

Judicial Case Management System (JCMS)

Production-Grade Blueprint & Execution Guide

1. Feature Scope & Functional Requirements

1.1 Judge Dashboard

Feature	Description
Hearing Docket View	Daily/weekly cause list auto-sorted by priority score; drag-to-reorder within permitted bounds
Case Disposition	Record orders, adjournments, next hearing dates, final judgments; attach signed order PDF
Courtroom Management	View/allocate courtroom + time slot; flag conflicts (double-booking, judge unavailability)
Case File Viewer	Consolidated view of petition, evidence, prior orders, and full timeline for a case
Emergency Queue Alerts	Real-time badge/notification when an urgent petition is filed and auto-routed to their bench
Bench Notes	Private judge-only notes per case (not visible to advocates/registrar)

1.2 Advocate/Lawyer Portal

Feature	Description
e-Filing	Submit new petitions/case filings with structured metadata (case type, parties, relief sought)
Evidence & Document Upload	Multi-file upload (PDF/images), versioned, virus-scanned, checksum-verified
Emergency Petition Flag	Mark a filing as urgent with justification; triggers expedited registrar review
Case Tracking	Status timeline: Filed → Under Verification → Listed → Heard → Disposed
Hearing Calendar	Personal calendar of all cases across courts, with conflict warnings for double-booked slots
Notifications	Email/SMS/push for hearing date changes, registrar objections, orders passed

1.3 Registrar/Court Staff Portal

Feature	Description
Filing Verification	Checklist-driven scrutiny (fee paid, format compliant, jurisdiction correct); accept/return-for-correction
Scheduling Engine Console	Run/override the auto-scheduler; manually resolve conflicts it flags
Queue Management	Visual priority queue per court/judge; manual re-prioritization with mandatory justification (logged)
Emergency Override Workflow	Fast-track intake → immediate registrar triage → same-day/next-slot bench assignment
Cause List Publishing	Generate and publish the next day's cause list (PDF + web)
Staff Task Queue	Assigned verification/administrative tasks with SLA timers

1.4 Public/Litigant Portal

Feature	Description
Public Cause List	Searchable, filterable, no-login list of hearings by court/date
Case Status Lookup	Search by case number/CNR-equivalent to see current stage (no sensitive documents exposed)
Order/Judgment Access	Download publicly available final orders (redacted per privacy rules)
Court/Judge Directory	Static reference info (courtroom numbers, sitting hours)

1.5 Cross-Cutting Standout Features

- Weighted Priority Scheduling Algorithm** — composite score from: case age (aging bonus), statutory urgency class (e.g., bail, habeas corpus > civil), emergency flag, court-imposed deadlines, and litigant vulnerability flags (e.g., senior citizen, POCSO). Recomputed on every schedule run; fully explainable (score breakdown visible to registrar/judge).
- Emergency Case Override Workflow** — a state machine: Filed(Emergency) → Registrar Triage (SLA: 2h) → Judge Acceptance → Slot Injection (bumps/reflows queue) → Notification fan-out. Every override is logged with actor, reason, and before/after queue snapshot.
- Digital Audit History** — immutable, append-only audit log (hash-chained) for every state-changing action: filings, verifications, reassignments, order uploads, overrides. Exportable per-case as a tamper-evident PDF trail.

2. Architecture & Tech Stack Recommendations

2.1 Recommended Stack

Layer	Choice	Why
Frontend	React (Vite) + TypeScript + TailwindCSS + shadcn/ui	Fast dev loop, strong typing, component reuse across 4 portals via shared design system
State/data	TanStack Query + Zustand	Server cache + minimal client state, avoids Redux boilerplate
Backend	Node.js + NestJS (TypeScript)	Opinionated modular architecture (great for role-based portals + DI + guards for RBAC), or Spring Boot (Java) if you want to signal enterprise/backend depth for recruiters targeting Java shops
Database	PostgreSQL	Strong relational integrity for case/hearing relationships; supports JSONB for flexible metadata, row-level security for RBAC enforcement at DB layer
ORM	Prisma (Node) or JPA/Hibernate (Java)	Type-safe schema, migrations
Auth	JWT (access + refresh) via Keycloak or Auth0 , or self-rolled with Passport.js + bcrypt/argon2	Keycloak looks great on a resume (OIDC, RBAC federation) and demonstrates real IAM knowledge
Caching	Redis	Cache hot cause-lists, session store, rate limiting, pub/sub for live updates
Queue	Redis Streams / BullMQ , or RabbitMQ for a more "enterprise" signal	Async jobs: document virus scan, PDF generation, notification fan-out, priority recompute
Real-time	WebSockets (Socket.IO) or Server-Sent Events	Live docket updates, emergency alerts, queue reordering pushed to judges/registrars instantly
File storage	S3-compatible (AWS S3 / MinIO for local dev)	Evidence/document storage with signed URLs, not DB blobs
Search	PostgreSQL full-text search (MVP) → Elasticsearch/OpenSearch (Phase 3)	Public case-status lookup and advocate search at scale

Layer	Choice	Why
Infra	Docker Compose (dev) → Kubernetes/ECS (prod), Terraform for IaC	Cloud-ready signal for recruiters
CI/CD	GitHub Actions	Lint/test/build/deploy pipeline, container publish
Observability	Prometheus + Grafana, Sentry for error tracking	Production-readiness signal

2.2 Relational Database Schema (Core Entities)

```
User (id, name, email, password_hash,  
role[Judge|Advocate|Registrar|Admin|Public], bar_council_id?, created_at)  
  
Court (id, name, location, jurisdiction_type)  
  
Courtroom (id, court_id FK, room_number, capacity)  
  
Judge (id, user_id FK, court_id FK, designation, is_active)  
  
Case (id, case_number [unique, indexed], case_type, filing_advocate_id FK-  
>User,  
      status[Filed|Verification|Returned|Listed|Heard|Disposed],  
      priority_score (computed, indexed), is_emergency (bool),  
      filed_at, court_id FK, assigned_judge_id FK->Judge nullable)  
  
Party (id, case_id FK, name, role[Petitioner|Respondent], contact_info)  
  
Hearing (id, case_id FK, courtroom_id FK, judge_id FK, scheduled_at,  
          status[Scheduled|Completed|Adjourned|Cancelled], order_summary,  
          next_hearing_date)  
  
Document (id, case_id FK, uploaded_by FK->User,  
           doc_type[Petition|Evidence|Order|Affidavit],  
           storage_url, version, checksum, virus_scan_status, uploaded_at)  
  
PriorityFactor (id, case_id FK,  
                factor_type[Age|StatutoryUrgency|EmergencyFlag|VulnerabilityFlag],  
                weight, computed_score, computed_at)  
  
AuditLog (id, entity_type, entity_id, action, actor_id FK->User, before_state  
JSONB,  
           after_state JSONB, reason, prev_hash, hash, created_at)  -- hash-  
chained for tamper evidence  
  
Notification (id, user_id FK, case_id FK, type, channel[Email|SMS|Push],  
              sent_at, read_at)
```

Key relationships: Case 1–N Hearing, Case 1–N Document, Case 1–N Party, Case 1–N PriorityFactor, Court 1–N Courtroom, Court 1–N Judge, every mutating action → 1 AuditLog row.

2.3 Caching / Queue Strategy

- **Redis cache-aside** for: today's cause list per court (TTL 5 min, invalidated on any hearing mutation), public case-status lookups.
 - **Priority queue** implemented as a Redis **sorted set** (`(ZADD court:{id}:queue {priority_score} {case_id})`) for $O(\log N)$ inserts/reorders — this is a strong talking point in interviews (heap-backed priority queue at scale, vs. recomputing SQL `ORDER BY` on every request).
 - **BullMQ jobs**: `(recompute-priority)` (nightly + on-event), `(generate-cause-list-pdf)`, `(notify-fanout)`, `(document-virus-scan)`.
 - **WebSocket rooms** per court (`(court:{id})`) and per user — registrar reorders queue → emits `(queue:updated)` → judge dashboards patch in place without refetch.
-

3. Backend API Endpoint Design (REST)

Auth

POST /api/auth/login
POST /api/auth/refresh
POST /api/auth/logout

Cases

POST /api/cases (advocate: file new case)
GET /api/cases/:id (role-scoped detail)
GET /api/cases?status=&court=&q= (search/list, paginated)
PATCH /api/cases/:id/status (registrar: verify/return)
POST /api/cases/:id/emergency-flag (advocate/registrar: mark urgent + justification)
GET /api/cases/:id/audit-trail (immutable history)

Priority Queue

GET /api/courts/:courtId/queue (current sorted queue with score breakdown)
POST /api/courts/:courtId/queue/reorder (registrar override, requires reason)
POST /api/scheduler/recompute (trigger recompute job)

Hearings / Courtroom Allocation

POST /api/hearings (registrar/judge: schedule)
PATCH /api/hearings/:id (reschedule/adjourn/dispose)
GET /api/courtrooms/:id/availability

Documents

POST /api/cases/:id/documents (multipart upload -> presigned S3 flow)
GET /api/documents/:id/download (signed URL, access-checked)

Public

GET /api/public/cause-list?court=&date=
GET /api/public/case-status?caseNumber=

Realtime (Socket.IO namespaces)

/ws/court/:courtId -> queue:updated, hearing:scheduled, emergency:alert

Auth on every non-public route via JWT + role guard; row-level checks (e.g., advocate can only see their own filed cases; judge only their assigned court).

4. Standout Portfolio Enhancements

1. **Real-time WebSocket Hearing Dockets** — live queue reordering, emergency alerts pushed instantly to judge/registrar dashboards without polling. Demonstrates event-driven architecture.
2. **AI-Assisted Petition Categorization** — use an LLM (or a lightweight classifier) to auto-suggest `case_type` and urgency class from the filed petition text, with a human-in-the-loop confirm step. Strong "applied AI" resume line, and pairs naturally with the priority-scoring engine.
3. **Explainable Weighted Priority Engine** — expose the score breakdown (age + urgency + emergency + vulnerability weights) as a visual "why is this case #3" tooltip. Signals algorithmic design skill, not just CRUD.
4. **Tamper-Evident Audit Log** — hash-chained audit trail (each entry stores hash of previous entry), verifiable integrity check endpoint. Strong systems-design/security talking point.
5. **PDF Cause-List/Order Generation** — server-side PDF generation (e.g., Puppeteer or a templating lib) for daily cause lists and signed order exports.

Pick 2-3 to build deeply rather than all 5 shallowly — depth reads better in interviews than breadth.

5. Phase-by-Phase Development Plan

Phase 1 — MVP (4-6 weeks)

- Auth + RBAC (4 roles), basic Postgres schema, Prisma/JPA setup
- Advocate: file case, upload documents (local/S3)
- Registrar: verify/return filing, manual hearing scheduling
- Judge: view docket, record disposition
- Public: case status lookup, static cause list
- Basic audit log (no hashing yet)
- Deployed on a single Docker Compose stack (Postgres + API + frontend)

Phase 2 — Core Differentiators (4-6 weeks)

- Weighted priority scoring engine + Redis sorted-set queue
- Emergency override workflow (state machine + SLA timers)
- WebSocket live docket updates
- Hash-chained tamper-evident audit log + verification endpoint
- PDF cause-list generation job (BullMQ)
- Notification fan-out (email at minimum)
- Test coverage: unit (scoring logic), integration (API + DB), one E2E flow (Playwright/Cypress)

Phase 3 — Advanced, Cloud-Ready (4-6 weeks)

- AI-assisted petition categorization (LLM API integration, human confirm step)
 - Move search to Elasticsearch/OpenSearch for public lookup at scale
 - Kubernetes manifests (or ECS) + Terraform IaC; GitHub Actions CI/CD to a real cloud environment
 - Observability: Prometheus/Grafana dashboards, Sentry error tracking, structured logging
 - Load testing (k6) on the priority-queue and public-lookup endpoints; document results
 - Security pass: rate limiting, input validation hardening, dependency scanning (Dependabot/Snyk), signed URL expiry review
 - Polish: recorded demo video, architecture diagram, README with setup + design-decision write-up (this last part matters as much as the code for portfolio impact)
-

Suggested Repo Structure for the Write-Up

Recruiters skim READMEs — lead with an architecture diagram, the priority-scoring explanation, and a live demo link/GIF before implementation details.